EVENT RAKSHAK

Unit Test Code Snippets

Document Type: Technical Reference — Test Code Specimens
Framework: Java / JUnit 5 / Mockito / AssertJ | Angular 21 / Jasmine / Karma
Section Reference: SRS Section 7.4.4
Version: 1.0

This document contains all three production-grade unit test code specimens extracted from the Event Rakshak Software Requirements Specification (SRS), Section 7.4.4. Each snippet is provided as a complete, annotated test class ready for integration into the project test suite.

Snippet 1 — JUnit 5: RiskCalculationServiceTest.java

Layer: Backend (Java) | Framework: JUnit 5 + Mockito + AssertJ | Class Under Test: RiskCalculationService

Validates the Music Concert risk strategy, indoor event guard (WeatherRiskService must not be invoked for indoor events), and the premium calculation formula. Uses @InjectMocks to inject a mocked WeatherRiskService into the real RiskCalculationService instance.

```java
// ================================================================
// RiskCalculationServiceTest.java  —  Event Rakshak Backend Tests
// ================================================================
package org.eventrakshak.service;
import org.eventrakshak.entity.Event;
import org.eventrakshak.entity.Policy;
import org.eventrakshak.enums.EventType;
import org.eventrakshak.service.risk.WeatherRiskService;
import org.junit.jupiter.api.*;
import org.mockito.*;
import static org.assertj.core.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;
class RiskCalculationServiceTest {
    @Mock
    private WeatherRiskService weatherRiskService;
    @InjectMocks
    private RiskCalculationService riskCalculationService;
    @BeforeEach
    void setUp() { MockitoAnnotations.openMocks(this); }
    // ■■ Helper: build a music event ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    private Event buildMusicEvent(int attendees, boolean fireNoc,
                                  boolean pyro, boolean outdoor) {
        Event e = new Event();
        e.setEventType(EventType.OUTDOOR_MUSIC_CONCERT);
        e.setNumberOfAttendees(attendees);
        e.setHasFireNoc(fireNoc);
        e.setFireworksUsed(pyro);
        e.setIsOutdoor(outdoor);
        e.setHasMetalDetectors(false);
        return e;
    }
    @Test
    @DisplayName("Music: 1500 attendees + no FireNOC + pyro = 8.5%")
    void should_Return8Point5_When_HighRiskMusicConcert() {
        // Arrange
        Event event = buildMusicEvent(1500, false, true, true);
        when(weatherRiskService.getWeatherRisk(any())).thenReturn(0.0);
        // Act
        double risk = riskCalculationService.calculateDomainRisk(event);
        // Assert
        assertThat(risk).isCloseTo(8.5, within(0.001));
    }
    @Test
```

```
    @DisplayName("Indoor event: WeatherRiskService must not be called")
    void should_NotCallWeatherService_When_EventIsIndoor() {
        Event event = buildMusicEvent(500, true, false, false);
        riskCalculationService.calculateTotalRisk(event, new Policy());
        verify(weatherRiskService, never()).getWeatherRisk(any());
    }
    @Test
    @DisplayName("Premium = maxCoverage * totalRisk / 100")
    void should_CalculateCorrectPremium_When_ValidRiskAndCoverage() {
        Event event = buildMusicEvent(1500, false, true, true);
        Policy policy = new Policy();
        policy.setMaxCoverageAmount(500000.0);
        when(weatherRiskService.getWeatherRisk(any())).thenReturn(0.0);
        double premium = riskCalculationService.calculatePremium(event, policy);
        assertThat(premium).isCloseTo(42500.0, within(0.001));
        // 500000 * 8.5 / 100 = 42500
    }
    @Test
    @DisplayName("Outdoor: humidity>80% + rain>5mm adds 2.5% weather risk")
    void should_AddWeatherRisk_When_OutdoorHighHumidityAndRain() {
        Event event = buildMusicEvent(500, true, false, true);
        when(weatherRiskService.getWeatherRisk(any())).thenReturn(2.5);
        double totalRisk = riskCalculationService.calculateTotalRisk(
            event, new Policy());
        assertThat(totalRisk).isGreaterThan(2.5);
        verify(weatherRiskService, times(1)).getWeatherRisk(event);
    }
}
```

Snippet 2 — Mockito: ClaimServiceTest.java

Layer: Backend (Java) | Framework: JUnit 5 + Mockito ArgumentCaptor | Class Under Test: ClaimService

Validates claim state machine transitions (PENDING → APPROVED/REJECTED), the approved-amount business rule (cannot exceed claimed amount), and the notification side-effect. Uses @Captor to inspect the exact entity state at the moment it is passed to the repository.

```
// ================================================================
// ClaimServiceTest.java  —  Event Rakshak Backend Tests
// ================================================================
package org.eventrakshak.service;
import org.eventrakshak.entity.Claim;
import org.eventrakshak.entity.Notification;
import org.eventrakshak.enums.ClaimStatus;
import org.eventrakshak.exception.ClaimValidationException;
import org.eventrakshak.exception.InvalidClaimStateException;
import org.eventrakshak.repository.ClaimRepository;
import org.eventrakshak.repository.NotificationRepository;
import org.junit.jupiter.api.*;
import org.mockito.*;
import java.util.Optional;
import static org.assertj.core.api.Assertions.*;
import static org.mockito.Mockito.*;
class ClaimServiceTest {
    @Mock private ClaimRepository claimRepository;
    @Mock private NotificationRepository notificationRepository;
    @InjectMocks
    private ClaimService claimService;
    @Captor ArgumentCaptor<Claim>        claimCaptor;
    @Captor ArgumentCaptor<Notification> notifCaptor;
    private Claim pendingClaim;
    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
        pendingClaim = new Claim();
        pendingClaim.setClaimId(1L);
        pendingClaim.setClaimAmount(50000.0);
        pendingClaim.setStatus(ClaimStatus.PENDING);
    }
    @Test
    void should_ApproveClaim_When_ValidAmountAndPending() {
        when(claimRepository.findById(1L))
            .thenReturn(Optional.of(pendingClaim));
        when(claimRepository.save(any())).thenAnswer(i -> i.getArgument(0));
        claimService.approveClaim(1L, 40000.0, "Evidence verified");
        verify(claimRepository).save(claimCaptor.capture());
        Claim saved = claimCaptor.getValue();
        assertThat(saved.getStatus()).isEqualTo(ClaimStatus.APPROVED);
        assertThat(saved.getApprovedAmount()).isEqualTo(40000.0);
        assertThat(saved.getResolvedAt()).isNotNull();
    }
```

```
    @Test
    void should_Throw_When_ApprovedAmountExceedsClaimed() {
        when(claimRepository.findById(1L))
            .thenReturn(Optional.of(pendingClaim));
        assertThatThrownBy(() ->
            claimService.approveClaim(1L, 99999.0, "Over-approval"))
            .isInstanceOf(ClaimValidationException.class)
            .hasMessageContaining("cannot exceed claimed amount");
    }
    @Test
    void should_Throw_When_ClaimAlreadyApproved() {
        pendingClaim.setStatus(ClaimStatus.APPROVED);
        when(claimRepository.findById(1L))
            .thenReturn(Optional.of(pendingClaim));
        assertThatThrownBy(() ->
            claimService.approveClaim(1L, 20000.0, "dup"))
            .isInstanceOf(InvalidClaimStateException.class);
    }
    @Test
    void should_SaveNotification_When_ClaimApproved() {
        when(claimRepository.findById(1L))
            .thenReturn(Optional.of(pendingClaim));
        when(claimRepository.save(any())).thenAnswer(i -> i.getArgument(0));
        claimService.approveClaim(1L, 30000.0, "All docs OK");
        verify(notificationRepository).save(notifCaptor.capture());
        assertThat(notifCaptor.getValue().getMessage())
            .containsIgnoringCase("approved");
    }
}
```

Snippet 3 — Jasmine: login.component.spec.ts (Angular 21)

Layer: Frontend | Framework: Angular 21 / Jasmine / Karma / TestBed | Component: LoginComponent

Validates form validation states (invalid/valid), service invocation on submit, navigation to /dashboard on success, and DOM error message rendering on 401 response. AuthService and Router are both replaced with Jasmine spy objects.

```
// ================================================================
// login.component.spec.ts  —  Event Rakshak (Angular 21)
// ================================================================
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { ReactiveFormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { By } from '@angular/platform-browser';
import { of, throwError } from 'rxjs';
import { LoginComponent } from './login.component';
import { AuthService } from '../../core/services/auth.service';
describe('LoginComponent', () => {
  let component: LoginComponent;
  let fixture: ComponentFixture<LoginComponent>;
  let authServiceSpy: jasmine.SpyObj<AuthService>;
  let routerSpy: jasmine.SpyObj<Router>;
  beforeEach(async () => {
    authServiceSpy = jasmine.createSpyObj('AuthService',
      ['login', 'isAuthenticated']);
    routerSpy = jasmine.createSpyObj('Router', ['navigate']);
    await TestBed.configureTestingModule({
      declarations: [LoginComponent],
      imports: [ReactiveFormsModule],
      providers: [
        { provide: AuthService, useValue: authServiceSpy },
        { provide: Router,      useValue: routerSpy },
      ],
    }).compileComponents();
    fixture = TestBed.createComponent(LoginComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });
  // ■■ Form Validation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  it('should be INVALID when email and password are empty', () => {
    expect(component.loginForm.valid).toBeFalse();
  });
  it('should mark email invalid for malformed address', () => {
    component.loginForm.get('email')!.setValue('bad-email');
    component.loginForm.get('password')!.setValue('Password@1');
    expect(component.loginForm.get('email')!.hasError('email')).toBeTrue();
  });
  it('should be VALID with correct email and password', () => {
    component.loginForm.setValue({
      email: 'admin@eventrakshak.com', password: 'Secure@1234'
    });
```

```
    expect(component.loginForm.valid).toBeTrue();
  });
  // ── Service Interaction ──────────────────────────────────────────────
  it('should NOT call AuthService when form is invalid', () => {
    component.onSubmit();
    expect(authServiceSpy.login).not.toHaveBeenCalled();
  });
  it('should call login() with form values on valid submit', () => {
    component.loginForm.setValue({
      email: 'admin@eventrakshak.com', password: 'Secure@1234'
    });
    authServiceSpy.login.and.returnValue(of({ token: 'mock.jwt' }));
    component.onSubmit();
    expect(authServiceSpy.login)
      .toHaveBeenCalledOnceWith('admin@eventrakshak.com', 'Secure@1234');
  });
  it('should navigate to /dashboard on successful login', () => {
    component.loginForm.setValue({
      email: 'admin@eventrakshak.com', password: 'Secure@1234'
    });
    authServiceSpy.login.and.returnValue(of({ token: 'tok' }));
    component.onSubmit();
    expect(routerSpy.navigate).toHaveBeenCalledWith(['/dashboard']);
  });
  // ── DOM Assertions ───────────────────────────────────────────────────
  it('should display error on 401 Unauthorized', () => {
    component.loginForm.setValue({
      email: 'bad@email.com', password: 'Wrong'
    });
    authServiceSpy.login.and.returnValue(
      throwError(() => ({ status: 401 })));
    component.onSubmit();
    fixture.detectChanges();
    const errEl = fixture.debugElement
      .query(By.css('[data-testid="login-error"]'));
    expect(errEl).not.toBeNull();
    expect(errEl.nativeElement.textContent)
      .toContain('Invalid credentials');
  });
  it('should not show error on pristine form', () => {
    const errEl = fixture.debugElement
      .query(By.css('[data-testid="login-error"]'));
    expect(errEl).toBeNull();
  });
});
```